Running the same test against a relational database supporting sequences, the following results are being recorded:

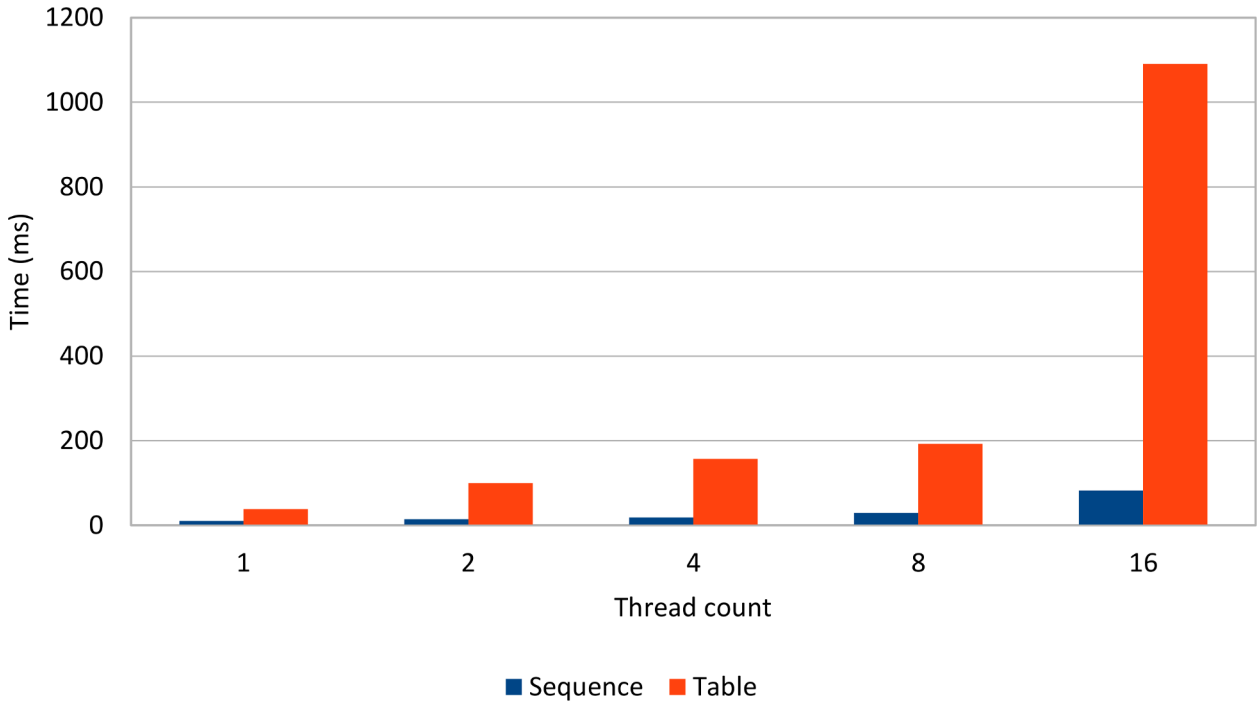

Figure 9.8: Sequence vs. Table

The performance impact of the table generator becomes noticeable in high concurrent environments, where the row-level locking and the database connection switch introduces a serial execution.

Because they use lightweight synchronization mechanisms, database sequences scale better than row-level locking concurrency control mechanisms.

Database sequences are the most efficient Hibernate identifier choice, allowing sequence call optimizers and without compromising JDBC batching.

10. Relationships

In a relational database, associations are formed by correlating rows belonging to different tables. A relationship is established when a child table defines a foreign key referencing the primary key of its parent table. Every database association is built on top of foreign keys, resulting three table relationship types:

- *one-to-many* is the most common relationship, and it associates a row from a parent table to multiple rows in a child table.
- *one-to-one* requires the child table Primary Key to be associated via a Foreign Key with the parent table Primary Key column.
- *many-to-many* requires a link table containing two Foreign Key columns that reference the two different parent tables.

The following diagram depicts all these three table relationships:

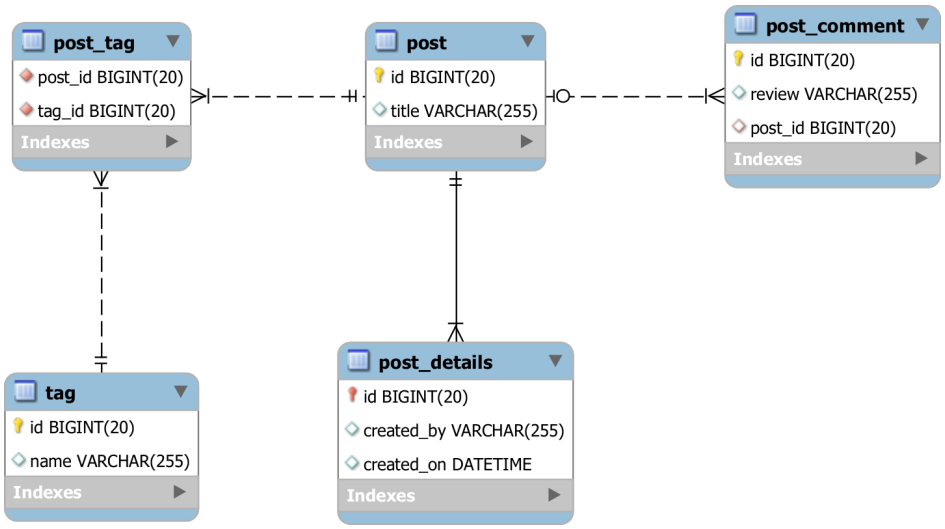

Figure 10.1: Table relationships

The `post` table has a one-to-many relationship with the `post_comment` table because a post row might be referenced by multiple comments. The one-to-many relationship is established through the `post_id` column which has a foreign key referencing the `post` table primary key. Because a `post_comment` cannot exist without a `post`, the `post` is the parent-side while the `post_comment` is the child-side.

The `post` table has a one-to-one relationship with the `post_details`. Like the one-to-many association, the one-to-one relationship involves two tables and a foreign key. The foreign key has a uniqueness constraint, so only one child row can reference a parent record.

The `post` and the `tag` are both independent tables and neither one is a child of the other. A `post` can feature several `tag(s)`, while a `tag` can also be associated with multiple `post(s)`. This is a typical many-to-many association, and it requires a junction table to resolve the child-side of these two parent entities. The junction table requires two foreign keys referencing the two parent tables.

The foreign key is, therefore, the most important construct in building a table relationship, and, in a relation database, the child-side controls a table relationship.

In a relational database, the foreign key is associated with the child-side only. For this reason, the parent-side has no knowledge of any associated child relationships, and, from a mapping perspective, table relationships are always unidirectional (the child foreign key references the parent primary key).

10.1 Relationship types

When mapping a JPA entity, besides the underlying table columns, the application developer can map entity relationships either in one direction or in a bidirectional way. This is another impedance mismatch between the object-oriented Domain Model and relational database system because, when using an ORM tool, the parent and the child-side can reference each other.

A relationship is unidirectional if only one entity side maps the table relationship and is bidirectional if the table relationship can be navigated in both directions (either from the entity parent-side or the child-side).

To properly represent both sides of an entity relationship, JPA defines four association mapping constructs:

- `@ManyToOne` represents the child-side (where the foreign key resides) in a database one-to-many table relationship.
- `@OneToMany` is associated with the parent-side of a one-to-many table relationship.
- `@ElementCollection` defines a one-to-many association between an entity and multiple value types (basic or embeddable).
- `@OneToOne` is used for both the child-side and the parent-side in a one-to-one table relationship.
- `@ManyToMany` mirrors a many-to-many table relationship.

Because the entity relationship choice has a considerable impact on the overall application performance, this chapter analyzes the data access operation efficiency of all these JPA associations.

Mapping collections

In a relational database, all table relationships are constructed using foreign keys and navigated through SQL queries. JPA allows mapping both the foreign key side (the child entity has a reference to its parent), as well as the parent side (the parent entity has one or more child entities).

Although `@OneToMany`, `@ManyToMany` or `@ElementCollection` are convenient from a data access perspective (entity state transitions can be cascaded from parent entities to children), they are definitely not free of cost. The price for reducing data access operations is paid in terms of result set fetching flexibility and performance. A JPA collection, either of entities or value types (basic or embeddables), binds a parent entity to a query that usually fetches all the associated child records. Because of this, the entity mapping becomes sensitive to the number of child entries.

If the children count is relatively small, the performance impact of always retrieving all child entities might be unnoticeable. However, if the number of child records grows too large, fetching the entire children collection may become a performance bottleneck. Unfortunately, the entity mapping is done during the early phases of a project development, and the development team might be unaware of the number of child records a production system exhibits.

Not just the mere size can be problematic, but also the number of attributes of the child entity. Because entities are usually fetched as a whole, the result set is, therefore, proportional to the number of columns the child table contains. Even if a collection is fetched lazily, Hibernate might still require to fully load each entity when the collection is accessed for the first time. Although Hibernate supports extra lazy collection fetching, this is only a workaround and does not address the root problem.

Alternatively, every collection mapping can be replaced by a data access query, which can use a SQL projection that is tailored to the data requirements of each business use case. This way, the query can take business case specific filtering criteria. Although JPA 2.1 does not support dynamic collection filtering, Hibernate offers Persistence Context-bound collection `Filters`.

When handling large data sets, it is good practice to limit the result set size, both for UI (to increase responsiveness) or batch processing tasks (to avoid long running transactions). Just because JPA offers supports collection mapping, it does not mean they are mandatory for every domain model mapping. Until there is a clear understanding of the number of child records (or if there is even need to fetch child entities entirely), it is better to postpone the collection mapping decision. For high-performance systems, a data access query is often a much more flexible alternative.

10.2 @ManyToOne

The `@ManyToOne` relationship is the most common JPA association, and it maps exactly to the one-to-many table relationship. When using a `@ManyToOne` association, the underlying foreign key is controlled by the child-side, no matter the association is unidirectional or bidirectional.

This section focuses on unidirectional `@ManyToOne` relationships only, the bidirectional case being further discussed with the `@OneToMany` relationship. In the following example, the `Post` entity represents the parent-side, while the `PostComment` is the child-side.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-many table relationship.

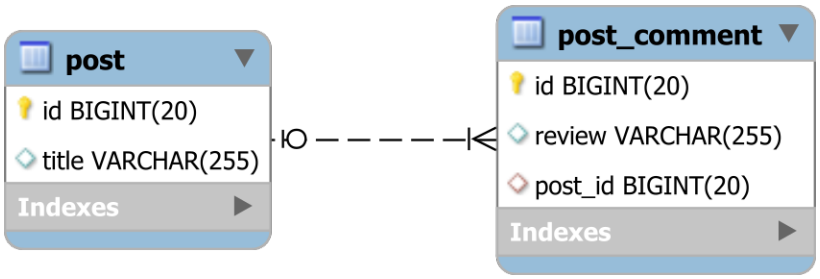

Figure 10.2: The one-to-many table relationship

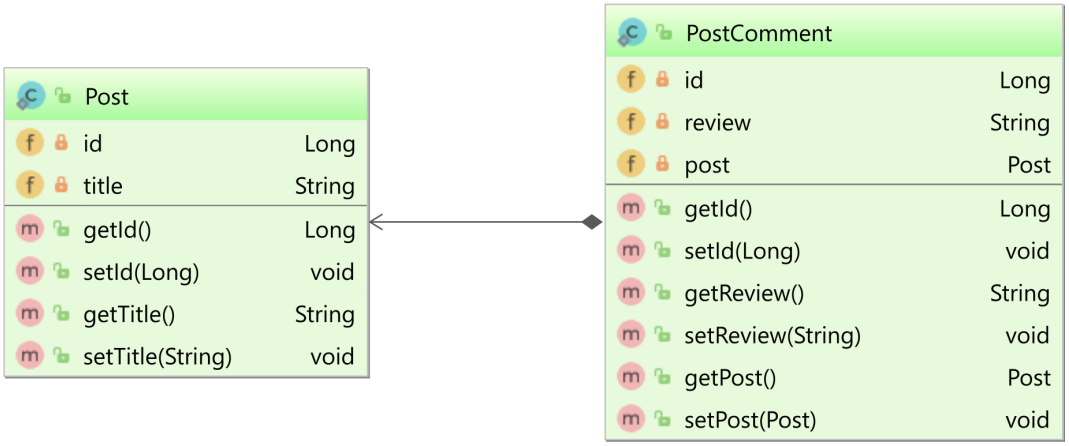

Figure 10.3: @ManyToOne relationship

Instead of mapping the `post_id` foreign key column, the `PostComment` uses a `@ManyToOne` relationship to the parent `Post` entity. The `PostComment` can be associated with an existing `Post` object reference, and the `PostComment` can also be fetched along with the `Post` entity.

```
@ManyToOne
@JoinColumn(name = "post_id")
private Post post;
```

Hibernate translates the internal state of the `@ManyToOne Post` object reference to the `post_id` foreign key column value.